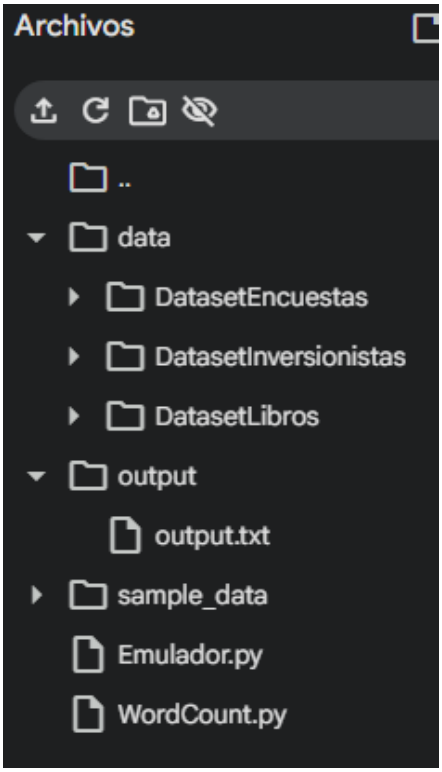
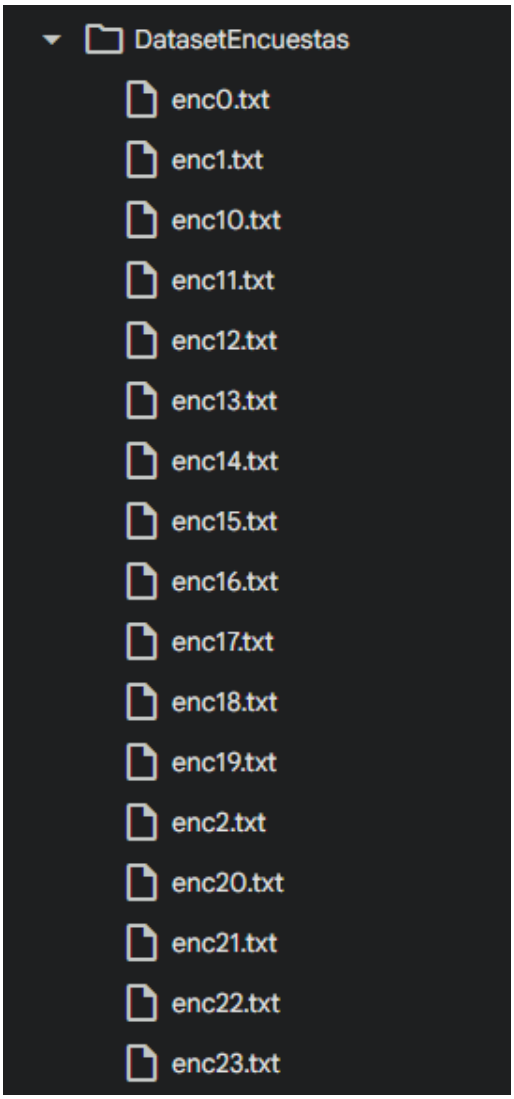
Practica 1

Para hacer las praticas en Google Collab

Estructura base



En cada dataset tengo todos los archivos individuales



Cambia mas que nada el archivo.py en que ejecutamos el emulador

Ejemplo de WordCount

```
from Emulador import Job

inputDir = "/content/data/DatasetEncuestas"
outputDir = "/content/output"

def fmap(key, value, context):
    words = value.split()
```

```
    for w in words:
        context.write(w, 1)

def fred(key, values, context):
    c=0
    for v in values:
        c=c+1
    context.write(key, c)

job = Job(inputDir, outputDir, fmap, fred)
success = job.waitForCompletion()
```

Ejercicio 1 (preguntar)

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	21	23	45	3	21	30	91
21	34	12	12	15	10	31	32
10	18	36	18	14	18	32	53
32	45	4	97	3	15	19	35

Que determina cuantas veces se ejecutan los maps y los reduce (PREGUNTAR)

1. Función `map` e Invocaciones

Cantidad de Mappers (instancias/nodos): Depende de la cantidad de **Splits** (bloques o particiones de entrada). En el ejercicio 1 hay 4 splits, por lo que se ejecutan **4 mappers**.

Invocaciones de la función `map` : Se ejecuta **una vez por cada registro (par clave-valor)** del dataset de entrada.

- En la tabla hay 4 registros por split x 4 splits = **16 registros totales**.
- Por lo tanto, la función `map` se invoca **16 veces** en todos los jobs del ejercicio 1.

2. Función `reduce` e Invocaciones

Invocaciones de la función `reduce`: Se ejecuta **exactamente una vez por cada clave intermedia única (`k2`)** emitida por la etapa de `map` tras el *Shuffle & Sort*.

Cantidad de Reducers (instancias/nodos): Viene dada por la configuración del cluster (por ejemplo, `setNumReducers(n)`). Sin embargo, una instancia de reducer solo trabajará si tiene al menos una partición/clave asignada.

Datos que recibe `reduce`: Recibe la clave intermedia agrupada (`k2`) y la lista o iterador con **todos los valores asociados a esa clave** emitidos por los mappers (`[v1, v2, ...]`).

Para determinar cantidad de mappers mirar cantidad de Splits, pero que son?

Lo mismo con reducers, se invocan por cada clave intermediaria unica emitida por el map

> Mira qué emite la función `map` en su primer argumento: `context.write(clave, valor)`. Cuenta cuántos valores distintos toma esa `clave` a lo largo de todos los datos procesados. **Ese número es la cantidad de invocaciones de `reduce`.**

```
En resumen
1) La cantidad de mappers depende de la cantidad de splits o
de nodos
2) La cantidad de veces que se ejecuta la funcion map es por
cada tupla clave
   valor
3) La cantidad de veces que se ejecuta la funcion reduce es p
or cada clave
   intermedia que le llega del map
4) La cantidad de reducers no los sabemos (por ahora)
5) Y el resultado del job depende map, el reduce y el algorit
mo de los mismos
```

Job A

a) Job A

```
def map(k1, v1, context):
      context.write(1, v1)

def reduce(k2, v2, context):
      n = 0
      for v in v2:
            n = n + 1
      context.write(k2, n)
```

Invocaciones de funcion Map: 1 x cada uno de los 16 registros de la tabla = 16

Invocaciones de reduce: 1 sola vez, por tener una clave unica

Cantidad de mappers: 4, son 4 por cada split

Cantidad de reducers: 1, porque siempre emite el num 1 como clave fija

Datos que recibe la funcion reduce: k2=1 y v2= todos los valores de la tabla de value

Salida del job: (1,16)

Job B

Invocaciones de funcion Map: igual que Job A

Invocaciones de reduce: igual que Job A

Cantidad de mappers: igual que Job A

Cantidad de reducers: igual que Job A

Datos que recibe la funcion reduce: k2= igual que Job A

Salida del job: (1,565) porque acumula los valores

Job C

Invocaciones de funcion Map: igual que Job A pero si v1 < 30 emite (1,k1) sino (2,k1)

Invocaciones de reduce: 2, 1 por cada clave

Cantidad de mappers:

Cantidad de reducers:

Datos que recibe la funcion reduce: para clave 1 los valores < 30 y paa clave 2 mayores a 30

Salida del job: para 1 - 36 y para 2 - 32

Job D

Invocaciones de funcion Map: igual que JOB A

Invocaciones de reduce: 13 veces por cada clave distinta

Cantidad de mappers: igual que Job A

Cantidad de reducers: 1

Datos que recibe la funcion reduce: 1

Salida del job:

Job E

Invocaciones de funcion Map: 16

Invocaciones de reduce: 12

Cantidad de mappers: 4

Cantidad de reducers: 1

Datos que recibe la funcion reduce:

Salida del job:

Ejercicio 2

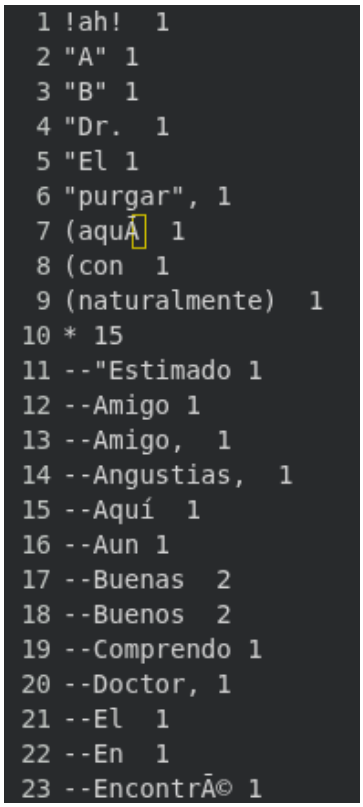
```
from Emulador import Job

inputDir = "/content/Data/Libros"
outputDir = "/content/output"

def fmap(key, value, context):
    words = value.split()
    for w in words:
        context.write(w, 1)
```

```python
def fred(key, values, context):
    c=0
    for v in values:
        c=c+1
    context.write(key, c)

job = Job(inputDir, outputDir, fmap, fred)
success = job.waitForCompletion()
```



```
 1 !ah!  1
 2 "A" 1
 3 "B" 1
 4 "Dr.  1
 5 "El 1
 6 "purgar", 1
 7 (aquÃ  1
 8 (con  1
 9 (naturalmente)  1
10 * 15
11 --"Estimado 1
12 --Amigo 1
13 --Amigo,  1
14 --Angustias,  1
15 --Aquí  1
16 --Aun 1
17 --Buenas  2
18 --Buenos  2
19 --Comprendo 1
20 --Doctor, 1
21 --El  1
22 --En  1
23 --EncontrÃ© 1
```

Y bastante mas

Ejercicio 3

Usar un segundo Job aprovechando el 1er WordCount

```
"""
Agrego 2do Job
ERA TEORICO Y ME COMPLIQUE AL PEDO

Ya no es un problema de big data,  el resultado del WordCount esta resumido
y se puede resolver con algoritmos de ordenamiento leyendo el archivo
"""
```

Ejercicio 4

Contar vocales, consonantes, digitos y otros valores

```
from Emulador import Job

inputDir = "/content/Data/Libros"
outputDir = "/content/output"

def fmap(key, value, context):
    for w in value:
        if w.isspace():
            context.write("espacios", 1)
        elif w.lower() in "aeiouáéíóú":
            context.write("vocales", 1)
        elif w.isalpha():
            context.write("consonantes", 1)
        elif w.isdigit():
            context.write("digitos", 1)
        else:
            context.write("otros", 1)
```

```
def fred(key, values, context):
    c=0
    for v in values:
        c=c+1
    context.write(key, c)

job = Job(inputDir, outputDir, fmap, fred)
success = job.waitForCompletion()
```

Ejercicio 5

Solo se pueden resolver aquellos relacionados con parrafos, puesto a que sabemos que terminan con un \n, pero no podemos determinar el resto.

Esto ultimo lo sabemos por como estan planteados los datasets de la catedra, en caso de tener otros se debe hacer un analisis de los mismos.

Ejercicio 6

```
from Emulador import Job

inputDir = "/content/Data/Encuestas"
outputDir = "/content/output"

def fmap(key, value, context):
    respuesta = value.strip().lower()

    categorias = {
        "muy satisfecho": "Muy satisfecho",
        "algo satisfecho": "Algo satisfecho",
        "poco satisfecho": "Poco satisfecho",
        "disconforme": "Disconforme",
        "muy disconforme": "Muy disconforme",
    }
```

```
    if respuesta in categorias:
        context.write(categorias[respuesta], 1)


def fred(key, values, context):
    c=0
    for v in values:
        c=c+1
    context.write(key, c)

job = Job(inputDir, outputDir, fmap, fred)
success = job.waitForCompletion()
```

Ejercicio 7 (arreglar)

```
from Emulador import Job
from datetime import date

inputDir = "C:/Users/Mar/Desktop/PracticaBigData/Data/Inversionistas"
outputDir = "C:/Users/Mar/Desktop/PracticaBigData/Output/"

## que no se note con a veces la hice en windows, otras en linux y otras en collab :)
## El nombre del inversionista más joven
## El total del importe invertido por todos los inversionistas
## El promedio de edad

def calcularMinimo(inversionista, edad, nombre, nuevaEdad):
    if(nuevaEdad<edad):
        edad= nuevaEdad
        inversionista= nombre
    return inversionista, edad
```

```
def CalcularEdad(year , month, day):
    hoy = date.today()
    edad = hoy.year - year - ((hoy.month, hoy.day) < (month,
day))
    return edad

def fmap(key, value, context):
    words = value.split()
    nombre= words[0]
    anio= int(words[3])
    mes= int(words[2])
    dia= int(words[1])
    aporte= float(words[4])
    context.write("GLOBAL", (nombre, anio, mes, dia, aporte))

def fred(key, values, context):
    totalAportes=0
    cantidadInversionistas=0
    edades=0
    inversionistaMasJoven=""
    edadMasJoven=100
    for v in values:
        cantidadInversionistas+=1
        totalAportes+=v[4]
        edadAux= CalcularEdad(v[1], v[2], v[3])
        edades+= edadAux
        inversionistaMasJoven, edadMasJoven = calcularMinimo
(inversionistaMasJoven, edadMasJoven, v[0], edadAux)

    promedio=edades/cantidadInversionistas
    context.write(key, ("Inversionista más joven: "+inversion
istaMasJoven, "Total aportes: "+str(totalAportes), "Promedio
de edad: "+str(promedio)))
```

```
job = Job(inputDir, outputDir, fmap, fred)
success = job.waitForCompletion()
```

Ejercicio 8

Deberia generar 33 claves intermerdias a mi eleccion, de manera que pueda distribuir la carga de trabajo entre todos los nodos.